

AI ENGINEERING · 30-DAY SPRINT

Job-Ready AI Portfolio Build Plan

Full SDLC Blueprint for 3 Production-Style AI Systems

PROJECT 1	PROJECT 2
RAG AI Chatbot (Document Q&A)	AI Resume Screener (Full-Stack SaaS)
PROJECT 3	TIMELINE
AI Agent Automation System	4 Weeks · ~10 hrs/week
DOCUMENT DATE	TARGET OUTCOME
June 2026	Internship / Junior AI Engineer Role

Table of Contents

Navigate the full build plan

1	Portfolio Strategy & Reality Check	04
2	Project 1 — RAG AI Chatbot (PDF / Docs Chat System)	05
3	Project 2 — AI Resume Screener (Full-Stack AI SaaS)	11
4	Project 3 — AI Agent Automation System	17
5	4-Week Combined Execution Timeline	23
6	Deployment & DevOps Checklist	24
7	Portfolio Presentation & LinkedIn Positioning	25

1. Portfolio Strategy & Reality Check

Why these 3 projects, and what "production-style" actually means

The goal of this plan is not to produce three disconnected demo scripts. It is to produce three systems that each demonstrate a distinct, in-demand AI engineering competency, built with the same level of rigor a company would expect from a junior engineer's first sprint. Each project below follows the same Software Development Life Cycle (SDLC) structure — Requirements → Design → Implementation → Testing → Deployment → Documentation — so that the process itself becomes part of what you can speak to in an interview.

Project	Core Skill Demonstrated	Interview Talking Point
RAG Chatbot	LLM orchestration, vector search, embeddings	"I built a retrieval pipeline and understand chunking/embedding trade-offs"
Resume Screener	Full-stack delivery, NLP, applied ML	"I shipped a usable SaaS product end-to-end, not just a notebook"
AI Agent System	Tool-using agents, orchestration logic, autonomy	"I understand agentic workflows beyond simple prompt-response"

Reality check: Three projects that "technically run" are not enough on their own — recruiters skim GitHub repos for evidence of testing, error handling, README quality, and deployment, not just a working demo on your laptop. This plan treats **testing and deployment as first-class steps**, not afterthoughts, so each project reads as production-style rather than tutorial-style.

What "Production-Style" Means Here

- **Version control discipline** — meaningful commits, a clear README, a LICENSE file, and a `.env.example` instead of hardcoded secrets.
- **Testing** — at minimum unit tests for core logic (chunking, scoring, parsing) and one integration test per API endpoint.
- **Error handling** — invalid file uploads, empty queries, API timeouts, and rate limits are handled gracefully, not left to crash the app.
- **Deployment** — a live, shareable URL. A project that only runs locally is a script, not a product.
- **Documentation** — README with architecture diagram, setup steps, and a 60–90 second demo video or GIF.

2. Project 1 — RAG AI Chatbot

PDF / Document Q&A System using Retrieval-Augmented Generation

LLM Orchestration Vector Search FastAPI + Streamlit

2.1 Project Overview

Problem statement: Users frequently need to extract specific answers from long PDFs or notes without reading the entire document. This system lets a user upload a document and ask natural-language questions, with the AI grounding every answer in the actual document content rather than hallucinating from general knowledge.

Goal: A user uploads a PDF or text document. The system extracts and chunks the text, embeds it into a vector store, and at query time retrieves the most relevant chunks to feed into an LLM, which generates a grounded answer with traceability back to the source.

2.2 Tech Stack

Layer	Tool / Framework	Purpose
Language	Python 3.11+	Core implementation language
Orchestration	LangChain	Chains together loading, splitting, embedding, retrieval, generation
LLM Provider	OpenAI API (or Ollama for local/free)	Generates the final natural-language answer
Embeddings	OpenAI <code>text-embedding-3-small</code> / HuggingFace <code>sentence-transformers</code>	Converts text chunks into vectors
Vector Store	FAISS (local) or ChromaDB (persistent)	Stores and searches embeddings by similarity
PDF Parsing	PyPDF2 / pdfplumber	Extracts raw text from uploaded PDFs
Backend API	FastAPI	Exposes <code>/upload</code> and <code>/chat</code> endpoints
Frontend	Streamlit	Upload UI + chat interface
Testing	pytest, httpx	Unit + API integration tests
Deployment	Docker, Render / Streamlit Cloud	Public hosting

2.3 System Architecture

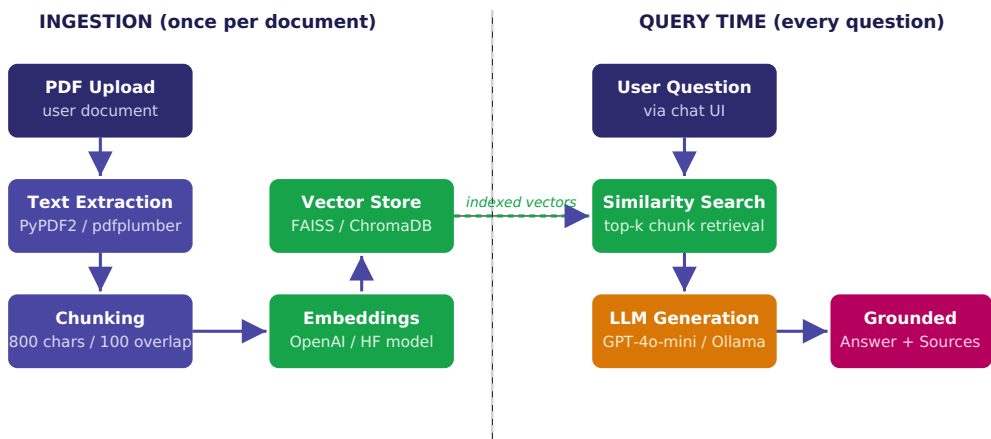


Figure 1 — RAG pipeline: ingestion (left) and query-time retrieval + generation (right)

2.4 Full SDLC Breakdown

Phase 1 — Requirements & Planning (Day 1)

Functional Requirements

- User can upload one PDF document (max 20MB) via UI.
- System extracts text and confirms successful ingestion.
- User can ask questions in a chat box and receive grounded answers.
- System shows which source chunk(s) the answer was based on (traceability).

Non-Functional Requirements

- Response latency under ~6 seconds for a typical query.
- Graceful failure on corrupted PDFs, scanned image-only PDFs, or empty documents.
- API keys never hardcoded — loaded via environment variables.

Phase 2 — Design (Day 1-2)

Repository Structure

```
ai-rag-chatbot/
├── backend/
│   ├── main.py          # FastAPI app entrypoint
│   ├── loaders.py       # PDF/text loading logic
│   ├── chunking.py      # Text splitting logic
│   ├── embeddings.py    # Embedding generation
│   ├── vectorstore.py   # FAISS/Chroma wrapper
│   ├── rag_chain.py     # Retrieval + generation pipeline
│   └── config.py        # Env var loading
├── frontend/
│   └── app.py           # Streamlit UI
├── tests/
│   ├── test_chunking.py
│   ├── test_loaders.py
│   └── test_api.py
├── .env.example
├── requirements.txt
├── Dockerfile
└── README.md
```

Data Flow (Sequence)

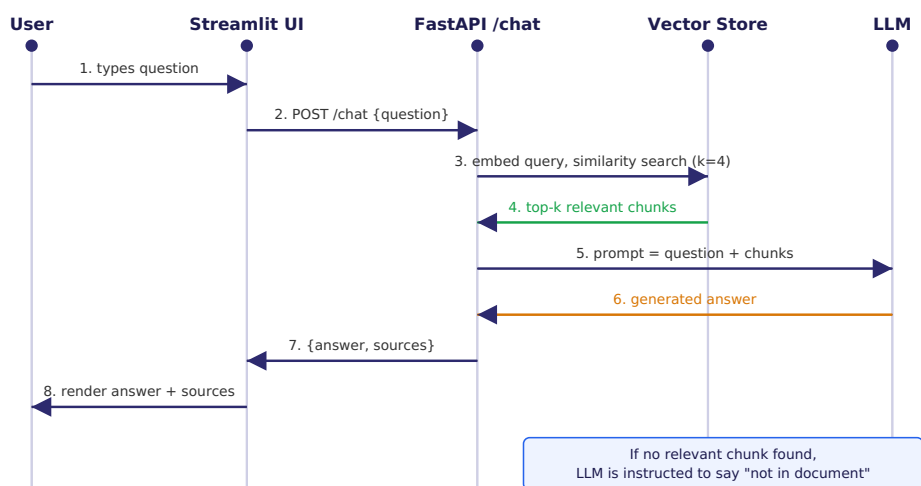


Figure 2 — Sequence diagram for a single user query

Phase 3 — Implementation (Day 2-6)

Step 1 — Environment Setup

```
python -m venv venv
source venv/bin/activate
pip install langchain langchain-community openai faiss-cpu \
    streamlit fastapi uvicorn pypdf python-dotenv
```

Step 2 — PDF Loader

```
from pypdf import PdfReader

def load_pdf(file_path: str) -> str:
    reader = PdfReader(file_path)
    text = ""
    for page in reader.pages:
        extracted = page.extract_text()
        if extracted:
            text += extracted + "\n"
```

```

if not text.strip():
    raise ValueError("No extractable text found — file may be scanned/image-only.")
return text

```

Edge case to handle: scanned PDFs return empty text. Detect this and either reject the upload with a clear error or fall back to OCR (pytesseract) as a stretch goal.

Step 3 — Text Chunking

```

from langchain.text_splitter import RecursiveCharacterTextSplitter

def chunk_text(text: str, chunk_size: int = 800, overlap: int = 100):
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=chunk_size,
        chunk_overlap=overlap
    )
    return splitter.split_text(text)

```

Design decision to document in your README: why 800/100 — smaller chunks improve retrieval precision but lose context; overlap prevents sentences being cut mid-thought across chunk boundaries.

Step 4 — Embeddings + Vector Store

```

from langchain_community.embeddings import OpenAIEmbeddings
from langchain_community.vectorstores import FAISS

def build_vectorstore(chunks: list[str]):
    embeddings = OpenAIEmbeddings(model="text-embedding-3-small")
    store = FAISS.from_texts(chunks, embeddings)
    store.save_local("faiss_index")
    return store

```

Step 5 — RAG Chain (Retrieval + Generation)

```

from langchain.chains import RetrievalQA
from langchain_community.chat_models import ChatOpenAI

def build_rag_chain(vectorstore):
    retriever = vectorstore.as_retriever(search_kwargs={"k": 4})
    llm = ChatOpenAI(model="gpt-4o-mini", temperature=0)
    return RetrievalQA.from_chain_type(
        llm=llm,
        retriever=retriever,
        return_source_documents=True
    )

```

Step 6 — FastAPI Endpoints

```

from fastapi import FastAPI, UploadFile, HTTPException
from pydantic import BaseModel

app = FastAPI()

class ChatRequest(BaseModel):
    question: str

@app.post("/upload")
async def upload_doc(file: UploadFile):
    if not file.filename.endswith(".pdf"):
        raise HTTPException(400, "Only PDF files are supported.")
    # save -> load_pdf -> chunk_text -> build_vectorstore
    return {"status": "indexed", "chunks": 42}

@app.post("/chat")
async def chat(req: ChatRequest):
    if not req.question.strip():
        raise HTTPException(400, "Question cannot be empty.")
    # run rag_chain
    return {"answer": "...", "sources": ["chunk_3", "chunk_7"]}

```

Step 7 — Streamlit UI

- File uploader widget → calls `/upload`
- Chat input + chat history using `st.session_state`
- Expandable "Sources" section under each answer showing retrieved chunks

Phase 4 — Testing (Day 7)

Test Type	What to Test	Tooling
Unit	Chunking produces correct chunk count/overlap; PDF loader raises on empty text	pytest
Unit	Embedding function returns correct vector dimensionality	pytest
Integration	<code>/upload</code> rejects non-PDF files; <code>/chat</code> rejects empty questions	pytest + <code>httpx.AsyncClient</code>
End-to-End	Upload a sample PDF → ask a known question → assert answer contains expected keyword	pytest + a fixture PDF
Manual / Exploratory	Ambiguous questions, questions with no answer in the doc (should say "not found", not hallucinate)	Manual

```
def test_chunk_text_overlap():
    text = "A" * 2000
    chunks = chunk_text(text, chunk_size=800, overlap=100)
    assert len(chunks) >= 2
    assert chunks[0][-100:] in chunks[1] # overlap exists

def test_upload_rejects_non_pdf(client):
    response = client.post("/upload", files={"file": ("notes.txt", b"hello")})
    assert response.status_code == 400
```

Phase 5 — Deployment (Day 8)

- **Containerize** backend with Docker; expose port 8000.
- **Backend host:** Render or Railway (free tier sufficient for portfolio use).
- **Frontend host:** Streamlit Community Cloud (free, directly deploys from GitHub).
- **Secrets:** set `OPENAI_API_KEY` as an environment variable in the host dashboard — never commit it.

```
# Dockerfile (backend)
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Phase 6 — Documentation (Day 8)

- ☐ README with: problem statement, architecture diagram, setup instructions, live demo link
- ☐ Short demo GIF showing upload → question → answer with sources
- ☐ "Design decisions" section explaining chunk size, retrieval k-value, and model choice trade-offs

2.5 Final Output

A live, shareable web app where a recruiter can upload their own PDF and chat with it — with retrieved source chunks visible, proving the system retrieves rather than hallucinates.

3. Project 2 — AI Resume Screener

Full-Stack AI SaaS: CV Analysis & Job Matching

Full-Stack NLP React + FastAPI + PostgreSQL

3.1 Project Overview

Problem statement: Job seekers don't know how well their resume matches a given role, and manually screening resumes is slow for recruiters. This system parses an uploaded CV, extracts structured information (skills, experience, education), and scores it against job descriptions to produce a match percentage and improvement suggestions.

Goal: A complete SaaS-style application with a React frontend, a FastAPI backend, an NLP-driven extraction/scoring engine, and persistent storage — demonstrating that you can ship a usable product, not just a script.

3.2 Tech Stack

Layer	Tool / Framework	Purpose
Frontend	React (Vite) + Tailwind CSS	Upload UI, results dashboard
Backend	FastAPI	REST API for upload, analysis, job matching
File Parsing	pdfplumber, docx2txt	Extracts raw text from PDF/DOCX resumes
NLP	spaCy (+ regex)	Named-entity extraction: skills, education, experience
Matching Engine	scikit-learn (TF-IDF + cosine similarity)	Scores resume against job description text
Database	PostgreSQL (via SQLAlchemy)	Stores users, resumes, results
Auth (optional)	JWT-based simple auth	Lets users save/revisit past analyses
Testing	pytest (backend), Vitest/RTL (frontend)	Unit + integration coverage
Deployment	Vercel (frontend), Render/Railway (backend), Supabase/Neon (DB)	Public hosting

3.3 System Architecture

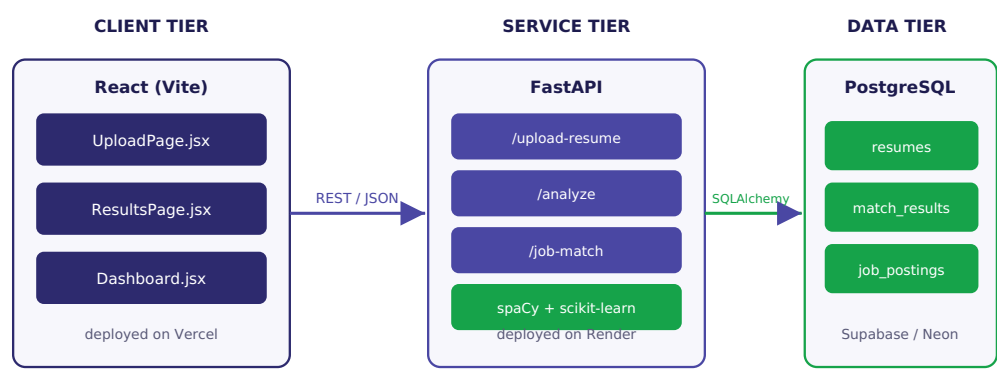


Figure 3 — Three-tier architecture: React client, FastAPI service layer, PostgreSQL persistence

3.4 Data Model (ER Diagram)

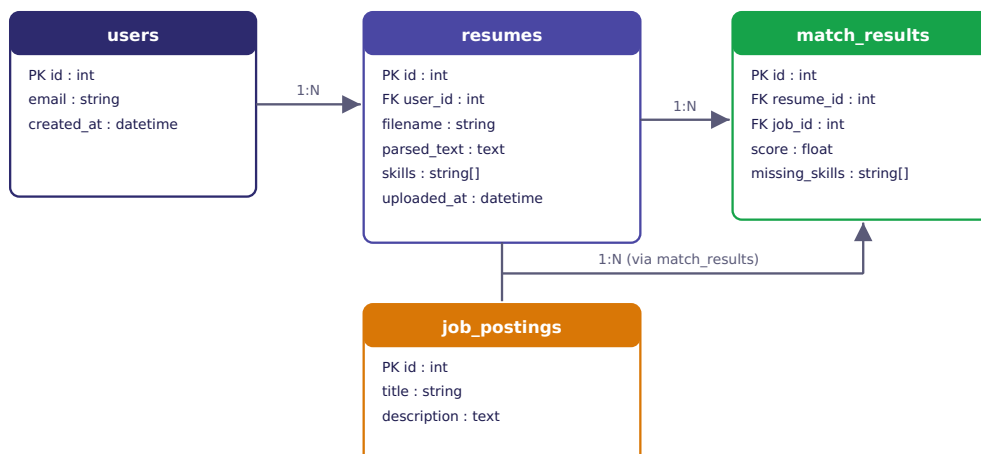


Figure 4 — Entity-relationship model for users, resumes, job postings, and match results

3.5 Full SDLC Breakdown

Phase 1 — Requirements & Planning (Day 9)

Functional Requirements

- User uploads a resume (PDF or DOCX).
- System extracts skills, experience duration, and education level.
- User pastes or selects a job description; system returns a match score (%) and a ranked skill-gap list.
- Results are persisted and viewable in a dashboard/history page.

Non-Functional Requirements

- Resume parsing must complete in under 3 seconds for files up to 5MB.
- Unsupported file types are rejected with a clear UI message, not a silent failure.
- No resume content is exposed to other users (basic data isolation per session/user).

Phase 2 — Design (Day 9-10)

Repository Structure

```

ai-resume-screener/
├── frontend/
│   ├── src/
│   │   ├── pages/UploadPage.jsx
│   │   ├── pages/ResultsPage.jsx
│   │   ├── pages/Dashboard.jsx
│   │   └── components/ResultCard.jsx
│   └── package.json
├── backend/
│   ├── main.py
│   ├── routers/upload.py
│   ├── routers/analyze.py
│   ├── routers/job_match.py
│   ├── services/parser.py          # pdfplumber/docx2txt extraction
│   ├── services/nlp_extract.py     # spaCy skill/entity extraction
│   ├── services/matcher.py         # TF-IDF + cosine similarity
│   ├── models.py                   # SQLAlchemy ORM models
│   └── database.py
├── tests/
│   ├── test_parser.py
│   ├── test_matcher.py
│   └── test_api.py
├── .env.example
└── README.md
  
```

API Design

Endpoint	Method	Description
<code>/upload-resume</code>	POST	Accepts file, returns parsed raw text + resume_id
<code>/analyze/{resume_id}</code>	GET	Returns extracted skills, education, experience

<code>/job-match</code>	POST	Body: resume_id + job description text → returns match score + gaps
<code>/history</code>	GET	Returns past analyses for the current user/session

Phase 3 — Implementation (Day 10-17)

Step 1 — Resume Text Extraction

```
import pdfplumber, docx2txt

def extract_text(file_path: str, content_type: str) -> str:
    if content_type == "application/pdf":
        with pdfplumber.open(file_path) as pdf:
            return "\n".join(page.extract_text() or "" for page in pdf.pages)
    elif "wordprocessingml" in content_type:
        return docx2txt.process(file_path)
    raise ValueError("Unsupported file type")
```

Step 2 — NLP Entity Extraction (spaCy)

```
import spacy
nlp = spacy.load("en_core_web_sm")

SKILL_KEYWORDS = {"python", "react", "sql", "docker", "fastapi", "aws", ...}

def extract_skills(text: str) -> list[str]:
    doc = nlp(text.lower())
    tokens = {token.text for token in doc}
    return sorted(SKILL_KEYWORDS.intersection(tokens))
```

Recommended approach (Option A from the original brief): start with keyword + regex matching for skills/education — it's transparent, debuggable, and good enough for a portfolio demo. TF-IDF + cosine similarity (Option B) is layered on top specifically for the resume-vs-job-description match score, not for entity extraction.

Step 3 — Job Match Scoring

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def compute_match_score(resume_text: str, job_text: str) -> float:
    vectorizer = TfidfVectorizer(stop_words="english")
    tfidf = vectorizer.fit_transform([resume_text, job_text])
    score = cosine_similarity(tfidf[0:1], tfidf[1:2])[0][0]
    return round(score * 100, 1) # percentage
```

Step 4 — FastAPI Routers

```
@router.post("/job-match")
async def job_match(payload: JobMatchRequest, db: Session = Depends(get_db)):
    resume = db.query(Resume).filter_by(id=payload.resume_id).first()
    if not resume:
        raise HTTPException(404, "Resume not found")
    score = compute_match_score(resume.parsed_text, payload.job_description)
    gaps = find_missing_skills(resume.skills, payload.job_description)
    save_match_result(db, resume.id, score, gaps)
    return {"match_score": score, "missing_skills": gaps}
```

Step 5 — React Frontend

- **UploadPage:** drag-and-drop zone (react-dropzone), upload progress, error toast on rejected file type.
- **ResultsPage:** circular progress indicator for match %, skill-gap chips, extracted-info summary card.
- **Dashboard:** table of past analyses pulled from `/history`.

Step 6 — Database Layer

```
class Resume(Base):
    __tablename__ = "resumes"
    id = Column(Integer, primary_key=True)
    filename = Column(String)
    parsed_text = Column(Text)
    skills = Column(ARRAY(String))
    uploaded_at = Column(DateTime, default=datetime.utcnow)

class MatchResult(Base):
```

```
__tablename__ = "match_results"
id = Column(Integer, primary_key=True)
resume_id = Column(Integer, ForeignKey("resumes.id"))
score = Column(Float)
missing_skills = Column(ARRAY(String))
```

Phase 4 — Testing (Day 18)

Test Type	What to Test	Tooling
Unit	Text extraction handles PDF, DOCX, and rejects unsupported types	pytest
Unit	Skill extraction returns expected skills from a fixture resume	pytest
Unit	Match score is deterministic and bounded 0-100	pytest
Integration	<code>/job-match</code> returns 404 for unknown resume_id	pytest + httpx
Frontend	Upload component renders error state on rejected file	Vitest + React Testing Library
End-to-End	Full flow: upload → analyze → job-match → dashboard shows new entry	Playwright (stretch goal)

```
def test_match_score_bounds():
    score = compute_match_score("python react sql", "python developer needed")
    assert 0 <= score <= 100

def test_job_match_unknown_resume(client):
    response = client.post("/job-match", json={"resume_id": 9999, "job_description": "x"})
    assert response.status_code == 404
```

Phase 5 — Deployment (Day 19)

Component	Host	Notes
Frontend (React)	Vercel	Auto-deploys from GitHub on push to main
Backend (FastAPI)	Render or Railway	Set <code>DATABASE_URL</code> as env var
Database	Supabase or Neon (free Postgres tier)	Run migrations via Alembic on first deploy

Phase 6 — Documentation (Day 19)

- ☐ README with architecture diagram + ER diagram
- ☐ API documentation (FastAPI auto-generates Swagger at `/docs` — link to it)
- ☐ Note the keyword-matching limitation honestly: "skill extraction uses a curated keyword list; a production version would use a fine-tuned NER model"

3.6 Final Output

A deployed CV analyzer where a user uploads a resume, pastes a job description, and instantly sees a match score, missing skills, and a saved history — a genuine full-stack AI SaaS product on your portfolio.

4. Project 3 — AI Agent Automation System

Autonomous Research & Report Agent

Agentic AI Tool Use LangGraph + FastAPI

4.1 Project Overview

Problem statement: A single prompt-response LLM call can't gather fresh information, decide which sources to consult, or produce a structured deliverable on its own. An agent that can reason about which tool to use, call it, observe the result, and decide on next steps demonstrates a meaningfully more advanced skill than basic prompting.

Goal (recommended scope): a Research + Report Agent. Given a topic, the agent searches the web, pulls relevant content, summarizes findings, and outputs a structured PDF/markdown report — autonomously, with minimal human steering after the initial request.

4.2 Tech Stack

Layer	Tool / Framework	Purpose
Language	Python 3.11+	Core implementation
Agent Orchestration	LangGraph (preferred) or LangChain Agents	Defines the agent's decision loop and state
LLM Provider	OpenAI API / Anthropic Claude API	Reasoning + summarization
Web Search Tool	Tavily API / SerpAPI	Lets the agent fetch live information
Browsing (optional)	Playwright	For agents that need to interact with pages, not just search
Memory	Redis (optional) or in-memory dict	Stores conversation/task state across steps
Backend API	FastAPI	Triggers agent runs, returns reports
Report Output	reportlab / markdown → PDF	Generates the final report file
Testing	pytest, mocked tool responses	Deterministic tests despite live API calls
Deployment	Docker + Render / AWS EC2	Public hosting

Important scoping note: "Email assistant" and "Crypto news analyzer" (the other options from the original brief) both introduce extra risk for a portfolio project — email assistants need OAuth and raise privacy concerns when demoed publicly, and crypto analysis invites questions about financial advice. The **Research + Report Agent** is the safest and most universally impressive choice for an internship-stage portfolio, which is why it's the recommended path here.

4.3 Agent Architecture & Decision Loop

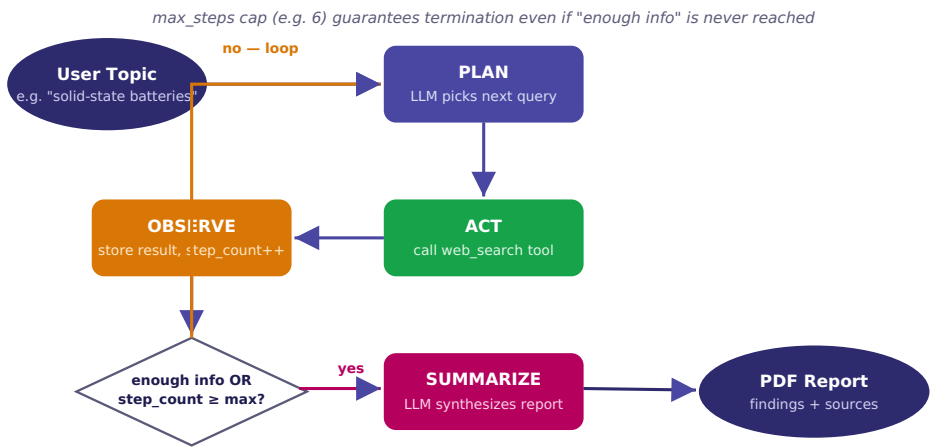


Figure 5 — Agent reasoning loop: plan → act (tool call) → observe → repeat until done

4.4 Full SDLC Breakdown

Phase 1 — Requirements & Planning (Day 20)

Functional Requirements

- User submits a research topic (e.g. "current state of solid-state batteries").
- Agent autonomously performs 2-5 searches, gathers relevant snippets, and synthesizes them.
- Agent outputs a structured report (introduction, key findings, sources, conclusion) as PDF or markdown.
- User can view run status (running / done / failed) and download the report.

Non-Functional Requirements

- Hard cap on tool calls per run (e.g. max 6) to prevent runaway loops and cost overruns.
- Timeout per run (e.g. 90 seconds) with a clear failure message if exceeded.
- All external tool calls logged for debuggability.

Phase 2 — Design (Day 20-21)

Repository Structure

```
ai-automation-agent/
├── agent/
│   ├── graph.py          # LangGraph state machine definition
│   ├── tools/
│   │   ├── web_search.py
│   │   ├── summarizer.py
│   │   └── file_writer.py
│   ├── state.py          # Agent state schema (Pydantic)
│   └── prompts.py
├── backend/
│   ├── main.py
│   └── routers/agent_run.py
├── tests/
│   ├── test_tools.py
│   ├── test_graph.py
│   └── test_api.py
├── reports/              # generated output, gitignored
├── .env.example
└── README.md
```

Agent State Schema

```
from pydantic import BaseModel

class AgentState(BaseModel):
    topic: str
    search_results: list[str] = []
    summary: str = ""
    step_count: int = 0
    max_steps: int = 6
    status: str = "running" # running | done | failed
```

Phase 3 — Implementation (Day 21-27)

Step 1 — Environment & LLM Setup

```
pip install langgraph langchain-openai tavily-python fastapi uvicorn reportlab python-dotenv
```

Step 2 — Web Search Tool

```
from tavily import TavilyClient
client = TavilyClient(api_key=os.environ["TAVILY_API_KEY"])

def web_search_tool(query: str, max_results: int = 5) -> list[dict]:
    results = client.search(query=query, max_results=max_results)
    return [{"title": r["title"], "content": r["content"], "url": r["url"]}
            for r in results["results"]]
```

Step 3 — Agent Graph (LangGraph)

```
from langgraph.graph import StateGraph, END

def plan_node(state: AgentState) -> AgentState:
    # LLM decides next search query based on topic + existing results
    ...
```

```
def search_node(state: AgentState) -> AgentState:
    results = web_search_tool(state.next_query)
    state.search_results.extend(results)
    state.step_count += 1
    return state

def should_continue(state: AgentState) -> str:
    if state.step_count >= state.max_steps or state.has_enough_info:
        return "summarize"
    return "plan"

graph = StateGraph(AgentState)
graph.add_node("plan", plan_node)
graph.add_node("search", search_node)
graph.add_node("summarize", summarize_node)
graph.add_conditional_edges("search", should_continue, {"plan": "plan", "summarize": "summarize"})
graph.add_edge("summarize", END)
graph.set_entry_point("plan")
agent = graph.compile()
```

Why LangGraph over a plain LangChain Agent: LangGraph makes the step limit and conditional routing explicit in the graph definition itself, which makes the runaway-loop safeguard easy to test and easy to explain in an interview.

Step 4 — Report Generation

```
from reportlab.platypus import SimpleDocTemplate, Paragraph
from reportlab.lib.styles import getSampleStyleSheet

def generate_report_pdf(topic: str, summary: str, sources: list[str], output_path: str):
    doc = SimpleDocTemplate(output_path)
    styles = getSampleStyleSheet()
    story = [
        Paragraph(f"Research Report: {topic}", styles["Title"]),
        Paragraph(summary, styles["Normal"]),
        Paragraph("Sources", styles["Heading2"]),
    ]
    for src in sources:
        story.append(Paragraph(src, styles["Normal"]))
    doc.build(story)
```

Step 5 — FastAPI Backend

```
@app.post("/run-agent")
async def run_agent(req: AgentRequest, background_tasks: BackgroundTasks):
    run_id = str(uuid4())
    background_tasks.add_task(execute_agent_run, run_id, req.topic)
    return {"run_id": run_id, "status": "running"}

@app.get("/get-report/{run_id}")
async def get_report(run_id: str):
    status = get_run_status(run_id)
    if status != "done":
        return {"status": status}
    return FileResponse(f"reports/{run_id}.pdf")
```

Step 6 — Optional Dashboard UI

- Streamlit or React page: text input for topic, "Run Agent" button, polling status indicator, download link once complete.
- Show the agent's intermediate steps (which queries it ran) for transparency — this is a strong demo feature.

Phase 4 — Testing (Day 28)

Test Type	What to Test	Tooling
Unit	Step-limit logic halts the graph at <code>max_steps</code>	pytest
Unit	Report generator produces a valid PDF with expected sections	pytest + pypdf to re-read output
Integration	Web search tool is mocked; agent still produces a coherent run with fixture data	pytest + unittest.mock
Integration	<code>/run-agent</code> returns a run_id immediately (non-blocking)	pytest + httpx

```
def test_agent_respects_max_steps():
    state = AgentState(topic="test", max_steps=2)
    final_state = run_graph_to_completion(state, mock_search=True)
    assert final_state.step_count <= 2

def test_search_tool_failure_marks_run_failed(monkeypatch):
    monkeypatch.setattr("agent.tools.web_search.web_search_tool",
                        lambda q: (_ for _ in ()).throw(ConnectionError()))
    result = execute_agent_run("test-run", "any topic")
    assert result.status == "failed"
```

Phase 5 — Deployment (Day 29)

- Containerize with Docker; run background agent tasks via FastAPI's `BackgroundTasks` or a lightweight queue (Celery + Redis) if you want to scale further.
- **Host:** Render (simplest) or AWS EC2 (better if you want to demo infra/DevOps skills too).
- Set `TAVILY_API_KEY` and LLM provider key as environment variables on the host.
- Rate-limit the public demo endpoint (e.g. 3 runs per IP per hour) to control API cost exposure.

Phase 6 — Documentation (Day 29-30)

- ☐ README explaining the agent loop with the decision-loop diagram from Section 4.3
- ☐ Example report output included in the repo as a sample artifact
- ☐ Clearly state safeguards: max steps, timeout, and rate limiting — this signals engineering maturity

4.5 Final Output

A live agent that takes a topic, autonomously researches it across multiple sources, and returns a downloadable structured report — your strongest "advanced AI thinking" portfolio piece.

5. 4-Week Combined Execution Timeline

~10 hours/week — realistic pace alongside interviews or coursework

Week	Days	Focus	Deliverable
1	1-8	Project 1 — RAG Chatbot	Deployed PDF Q&A app
2	9-19	Project 2 — Resume Screener	Deployed full-stack SaaS
3	20-29	Project 3 — AI Agent System	Deployed research agent
4	30	Polish, READMEs, demo videos, LinkedIn	Portfolio ready to share

Note on pacing: Project 2 is given more days (11 vs. 8) because full-stack work — wiring a React frontend to a FastAPI backend to a Postgres database — has more integration surface area than a single-service Python app. If you're already comfortable with React, you can compress this to 8 days and bank the extra time as buffer for Project 3's deployment debugging.

6. Deployment & DevOps Checklist

Apply this checklist to all three projects before calling any of them "done"

- ☐ Repository has a clear, single-purpose README (problem → architecture → setup → demo link)
- ☐ No secrets committed — `.env` is gitignored, `.env.example` is provided
- ☐ Dependencies pinned in `requirements.txt` / `package.json`
- ☐ At least one passing test suite, runnable with a single command (`pytest` , `npm test`)
- ☐ Error states handled in the UI — not just the happy path
- ☐ App is live at a public URL and was tested fresh (incognito browser) before sharing
- ☐ Basic rate limiting or cost guardrails on any endpoint that calls a paid LLM API
- ☐ A 60–90 second screen recording or GIF demonstrating the core flow

Cost control reminder: all three projects call paid LLM APIs. Set low usage caps/budget alerts in your OpenAI/Anthropic dashboard before deploying anything publicly, and consider swapping to a local model (Ollama) for the always-on portions of the demo if cost becomes a concern.

7. Portfolio Presentation & LinkedIn Positioning

How to talk about this work once it's built

7.1 Suggested LinkedIn Headline

AI Engineer | LLM | RAG | FastAPI | React | Python | AI Automation

7.2 How Each Project Maps to Interview Questions

Likely Question	Project to Reference
"Walk me through a RAG system you've built."	Project 1 — discuss chunking strategy, retrieval k-value, and how you prevented hallucination
"Tell me about a full-stack project you shipped end-to-end."	Project 2 — discuss the React↔FastAPI↔Postgres flow and the matching algorithm trade-offs
"Have you worked with AI agents or tool use?"	Project 3 — discuss the decision loop, step-limit safeguard, and failure handling
"How do you handle testing in your projects?"	Any — point to the pytest suites and the specific edge cases you covered

7.3 Final Honest Assessment

Scenario	Outcome
3 projects finished, but no tests, no deployment, no README	✗ Reads as tutorial-following, not engineering — not enough on its own
3 projects finished as production-style systems per this plan	✓ Strong, credible portfolio for an internship or junior AI engineering role

End of Build Plan — Good luck with the build, and with the interview.